



中南大學
CENTRAL SOUTH UNIVERSITY

嵌入式课程作业

学生姓名：樊博文

学 号：8207230322

学 院：自动化学院

专业班级：电子信息工程 2303

目录

5 级流水线运行 6 条指令需时间?	2
CPSR=0x03H 时的运行模式与状态	2
思考题 3-4 B、BL、BX 指令的区别?	2
思考题 3-8 编写一段 ARM 汇编程序, 实现数据块拷贝, 将 R0 指向的 8 个字的连续数据保存到 R1 指向的一段连续的内存单元。	3
编写相同功能 c 语言程序	3
编写相同功能 c 语言程序	4
分析 p88 例 2 每一步程序结果 计算 f(5)=	5
根据 LUT 设计 LED 跑马灯程序.....	6
同时按下 2 键的结果.....	7
课件 8.3.2 程序, 公式法和查表法求波特率和系统时钟	8
根据书 p201 主程序画流程图.....	10

5 级流水线运行 6 条指令需时间？

5 段流水线设计：

Inst0	IA	IF	DC	EX	WB					
Inst1		IA	IF	DC	EX	WB				
Inst2			IA	IF	DC	EX	WB			
Inst3				IA	IF	DC	EX	WB		
Inst3					IA	IF	DC	EX	WB	

在理想情况下，5 级流水线执行指令时，第一条指令需要经过 5 个时钟周期才能完成，之后每增加一条指令，只需要再增加 1 个时钟周期。

因此，总周期数为：

$$5 + (6 - 1) = 10$$

所以，5 级流水线运行 6 条指令共需要 **10 个时钟周期**。

CPSR=0x03H 时的运行模式与状态

CPSR 的低 5 位 M[4:0] 用来表示 ARM 处理器的工作模式。

$$0x03 = 00000011B$$

所以：

$$M[4:0] = 00011$$

该值不属于 ARM 中常见的有效工作模式编码，例如 User、FIQ、IRQ、SVC、Abort、Undefined、System 等模式，因此可判断为**无效或未定义工作模式**。

CPSR 中的 T 位用于判断处理器状态：

T = 0: ARM 状态；

T = 1: Thumb 状态。

由于 0x03 中 T 位为 0，因此处理器处于 **ARM 状态**。

思考题 3-4 B、BL、BX 指令的区别？

B 是普通跳转指令，只改变程序计数器 PC 的值，实现程序跳转。它不会保存返回地址，常用于无条件跳转或条件跳转。

BL 是带链接的跳转指令。执行 BL 时，处理器会把下一条指令的地址保存到链接寄存器 LR 中，然后跳转到目标地址执行。因此，BL 常用于子程序调用，子程序结束后可以通过 MOV PC, LR 或类似指令返回。

BX 是带状态切换功能的跳转指令。它根据目标地址最低位 bit[0] 判断跳转后的状态：如果 bit[0] 为 1，则进入 Thumb 状态；如果 bit[0] 为 0，则进入 ARM 状态。因此，BX 常用于 ARM 指令集和 Thumb 指令集之间的切换。

思考题 3-8 编写一段 ARM 汇编程序，实现数据块拷贝，将 R0 指向的 8 个字的连续数据保存到 R1 指向的一段连续的内存单元。

```
AREA DataCopy, CODE, READONLY
ENTRY

Start
    ; R0 为源数据块首地址
    ; R1 为目标数据块首地址
    ; 每个字为 32 位，共拷贝 8 个字

    LDMIA R0!, {R2-R9}    ; 从 R0 指向的地址连续读取 8 个字到 R2-R9
    STMIA R1!, {R2-R9}    ; 将 R2-R9 中的数据连续写入到 R1 指向的地址

Stop
    B      Stop           ; 程序停在此处

END
```

说明：

LDMIA R0!, {R2-R9} 表示从 R0 指向的内存地址开始，连续读取 8 个字，分别存入 R2 到 R9。由于使用了 !，执行后 R0 会自动增加，指向源数据块后面的地址。

STMIA R1!, {R2-R9} 表示将 R2 到 R9 中的 8 个字，连续写入到 R1 指向的目标内存区域中。执行后 R1 也会自动增加，指向目标数据块后面的地址。

因此，该程序完成了从源地址 R0 到目标地址 R1 的 8 个连续字的数据块拷贝。

编写相同功能 c 语言程序

```
IF R0=0x10
ADD R0,R1,R2
Else
ADD R0,R1,R3
ENDIF
```

对应逻辑为：

```
if (R0 == 0x10)
    R0 = R1 + R2;
else
    R0 = R1 + R3;
```

C 语言程序如下:

```
int func(int R0, int R1, int R2, int R3)
{
    if (R0 == 0x10)
    {
        R0 = R1 + R2;
    }
    else
    {
        R0 = R1 + R3;
    }

    return R0;
}
```

编写相同功能 c 语言程序

```
GBLA Con1
Con1 SETA
WHILE Cou1<10
ADD R1,R2,R3
Con1 SETA Con1+1
WEND
```

汇编表示循环执行 10 次加法操作，对应的 C 语言程序为：

```
int func(int R2, int R3)
{
    int R1 = 0;
    int Con1;

    for (Con1 = 0; Con1 < 10; Con1++)
    {
        R1 = R2 + R3;
    }

    return R1;
}
```

分析 p88 例 2 每一步程序结果 计算 $f(5)=$

据 ARM 调用规则, 函数参数通常通过 R0-R3 传递, 超过 4 个的参数通过栈传递, 函数返回值通过 R0 返回。

设进入函数 f 时:

$$R0 = i = 5$$

程序执行过程如下:

1. STR lr, [sp, #-4]!

保存返回地址 lr, 防止调用其他函数时返回地址被覆盖。

2. ADD R1, R0, R0

$$R1 = R0 + R0 = 5 + 5 = 10$$

即得到第二个参数:

$$2i = 10$$

3. ADD R2, R1, R0

$$R2 = R1 + R0 = 10 + 5 = 15$$

即得到第三个参数:

$$3i = 15$$

4. ADD R3, R1, R2

$$R3 = R1 + R2 = 10 + 15 = 25$$

即得到第五个参数:

$$5i = 25$$

由于第五个参数不能通过 R0-R3 传递, 所以需要压入栈中。

5. STR R3, [sp, #-4]!

将第五个参数 25 压入栈中。

6. ADD R3, R1, R1

$$R3 = R1 + R1 = 10 + 10 = 20$$

即得到第四个参数:

$$4i = 20$$

此时调用函数 g 时, 各参数为:

$$g(5, 10, 15, 20, 25)$$

7. BL g

调用函数 g, 假设函数 g 的功能是求 5 个参数之和, 则:

$$g(5, 10, 15, 20, 25) = 5 + 10 + 15 + 20 + 25 = 75$$

函数返回值保存在 R0 中, 因此:

$$R0 = 75$$

8. ADD sp, sp, #4

释放之前压入栈中的第五个参数。

最终结果为:

$$f(5) = 75$$

根据 LUT 设计 LED 跑马灯程序

```
#include "s5pc100.h"

typedef struct
{
    unsigned int GPG3CON;
    unsigned int GPG3DAT;
} GPG3_TypeDef;

#define GPG3 ((volatile GPG3_TypeDef *)0xE03001C0)

void delay(void)
{
    volatile unsigned int i;
    for (i = 0; i < 100000; i++);
}

/*
    LED 为低电平点亮:
    0x0E = 1110B, LED0 亮
    0x0D = 1101B, LED1 亮
    0x0B = 1011B, LED2 亮
    0x07 = 0111B, LED3 亮
*/
const unsigned char led_lut[] =
{
    0x0E,
    0x0D,
    0x0B,
    0x07,
    0x00,
    0x0F
};

#define LUT_LEN (sizeof(led_lut) / sizeof(led_lut[0]))

int main(void)
{
    unsigned int index = 0;

    /* 配置 GPG3_0 ~ GPG3_3 为输出功能 */
    GPG3->GPG3CON &= ~((0xF << 0) | (0xF << 4) | (0xF << 8) | (0xF << 12));
    GPG3->GPG3CON |= ((0x1 << 0) | (0x1 << 4) | (0x1 << 8) | (0x1 << 12));
}
```

```
while (1)
{
    GPG3->GPG3DAT = (GPG3->GPG3DAT & ~0x0F) | led_lut[index];

    index++;

    if (index >= LUT_LEN)
    {
        index = 0;
    }

    delay();
}

return 0;
}
```

同时按下 2 键的结果.

当 Key1 和 Key2 同时按下时，两个按键对应的外部中断标志位都会被置位。程序进入中断服务函数后，会按照程序中判断中断标志位的顺序依次处理两个中断。

如果程序先判断 Key1，再判断 Key2，则输出顺序一般为：

open uart device ok!

please press Key1 or Key2

In do_irq

in int key1

in int key2

如果按键存在机械抖动，或者程序没有完全消抖，可能会出现某个按键中断被触发多次的情况，例如：

In do_irq

in int key1

In do_irq

in int key1

In do_irq

in int key2

因此，同时按下两个按键时，理论结果是 Key1 和 Key2 的中断都会被响应，实际输出顺序取决于程序中对中断标志位的判断顺序。

课件 8.3.2 程序，公式法和查表法求波特率和系统时钟

已知：

$$UBRDIV0 = 34$$

$$UDIVSLOT0 = 0xDFDD$$

$$bps = 115200$$

1. 公式法

UART 分频公式为：

$$DIV_VAL = UBRDIVn + \frac{UDIVSLOTn \text{中} 1 \text{的个数}}{16}$$

同时：

$$DIV_VAL = \frac{PCLK}{bps \times 16} - 1$$

由 $UDIVSLOT0 = 0xDFDD$ 可知，其二进制中共有 13 个 1。

所以：

$$DIV_VAL = 34 + \frac{13}{16}$$

$$DIV_VAL = 34 + 0.8125 = 34.8125$$

由公式变形得：

$$PCLK = (DIV_VAL + 1) \times bps \times 16$$

代入数据：

$$PCLK = (34.8125 + 1) \times 115200 \times 16$$

$$PCLK = 35.8125 \times 115200 \times 16$$

$$PCLK = 66009600Hz$$

因此：

$$PCLK \approx 66MHz$$

若系统时钟近似取 66MHz，则波特率为：

$$bps = \frac{PCLK}{(DIV_VAL + 1) \times 16}$$

$$bps = \frac{66000000}{35.8125 \times 16}$$

$$bps \approx 115183bps$$

该值非常接近标准波特率 115200bps，误差很小，因此可认为波特率为：

$$bps \approx 115200bps$$

结论：

$$PCLK \approx 66MHz$$

$$bps \approx 115200bps$$

2. 查表法

查表法的关键是通过 $UDIVSLOT0$ 的取值快速得到小数分频部分。

已知：

$$UDIVSLOT0 = 0xDFDD$$

查表可知该值对应 13 个 1，因此：

$$\frac{13}{16} = 0.8125$$

又因为:

$$UBRDIV0 = 34$$

所以:

$$DIV_VAL = 34 + 0.8125 = 34.8125$$

再根据:

$$PCLK = (DIV_VAL + 1) \times bps \times 16$$

可得:

$$PCLK = 35.8125 \times 115200 \times 16$$

$$PCLK = 66009600Hz \approx 66MHz$$

因此, 查表法得到的结果与公式法一致:

$$PCLK \approx 66MHz$$

$$\text{波特率} \approx 115200bps$$

根据书 p201 主程序画流程图

